Instituto Superior de Engenharia de Lisboa
Programação III (PG3)
1º Teste – 4 de novembro de 2026 (duração: 1h15)

Considere um sistema de gestão de frotas de veículos (parecido com o do enunciado do primeiro trabalho prático, mas não a mesma coisa). O diagrama de classes em anexo representa um excerto desse sistema. O diagrama é constituído pela interface **`Rentable`** que define as operações principais sobre o aluguer de um veículo, pela classe abstrata **`Vehicle`** que define os membros que caracterizam genericamente um veículo de aluguer, e pelas classes **`CargoVan`** e **`Car`** que representam veículos concretos de aluguer. A classe **`Rental`** representa um contrato de aluguer, estando presente no sistema enquanto permanecer ativo. A classe **`RentalException`** representa uma exceção verificada ocorrida durante o processo de aluguer. No diagrama de classes, membros sublinhados são estáticos e características entre parêntesis significam: *final* - classe ou membro final; *IllegalArgumentException, RentalException* – exceções produzidas nos métodos; *abstract* - método abstrato; e literais numéricos - valores de constantes.

Considere as definições da interface **`Rentable`** e da classe **`RentalException`**:

```
public interface Rentable {
    void rent() throws RentalException; // aluga o veículo
    void returnVehicle() throws RentalException; // entrega do veículo alugado
}

public class RentalException extends Exception {
    public RentalException(String message) { super(message); }
    public RentalException() { super("Rental failed"); }
}
```

Nas suas implementações, considere que todos os parâmetros de tipo referência recebem valores diferentes de **`null`**, logo não são necessárias validações adicionais às explicitamente pedidas no enunciado. As duas operações de **`Rentable`** podem lançar **`RentalException`** revelando insucesso da operação. Não precisa implementar a classe **`Car`**.
Tendo em conta as considerações anteriores, o diagrama de classes em anexo e as definições da interface **`Rentable`** e da classe **`RentalException`**:

1. [5] Implemente a classe **`Vehicle`**, considerando os seguintes requisitos:

   - Os campos **`licensePlate`** e **`isAvailable`** identificam, respetivamente, a matrícula do veículo, cujo valor é especificado no parâmetro **`licensePlate`** do construtor, e se está disponível para ser alugado (a **`true`**) ou se está alugado (a **`false`**). Após construção, um veículo está disponível para ser alugado;
   - O método **`rent`** aluga o veículo, lançando **`RentalException`** com a mensagem *"Vehicle is already rented"* se o veículo já estiver alugado;
   - O método **`returnVehicle`** disponibiliza o veículo novamente para aluguer, lançando **`RentalException`** com a mensagem *"Vehicle is not rented"* se já estiver disponível para alugar;
   - O método **`calculateRental`** calcula o custo total do aluguer baseado no número de dias recebido no parâmetro **`days`** e na tarifa diária obtida pelo método **`getDailyRate`**. O método **`calculateRental`** produz a exceção não verificada **`IllegalArgumentException`** se o valor do parâmetro **`days`** for negativo;
   - Dois veículos são considerados iguais se as respetivas matrículas forem iguais;
   - O método **`toString`** é implementado com a instrução:
     `return "Vehicle{licensePlate=\"" + licensePlate + "\", isAvailable=" + isAvailable + '}';`

2. [3] Implemente a classe **`CargoVan`**, considerando os seguintes requisitos:

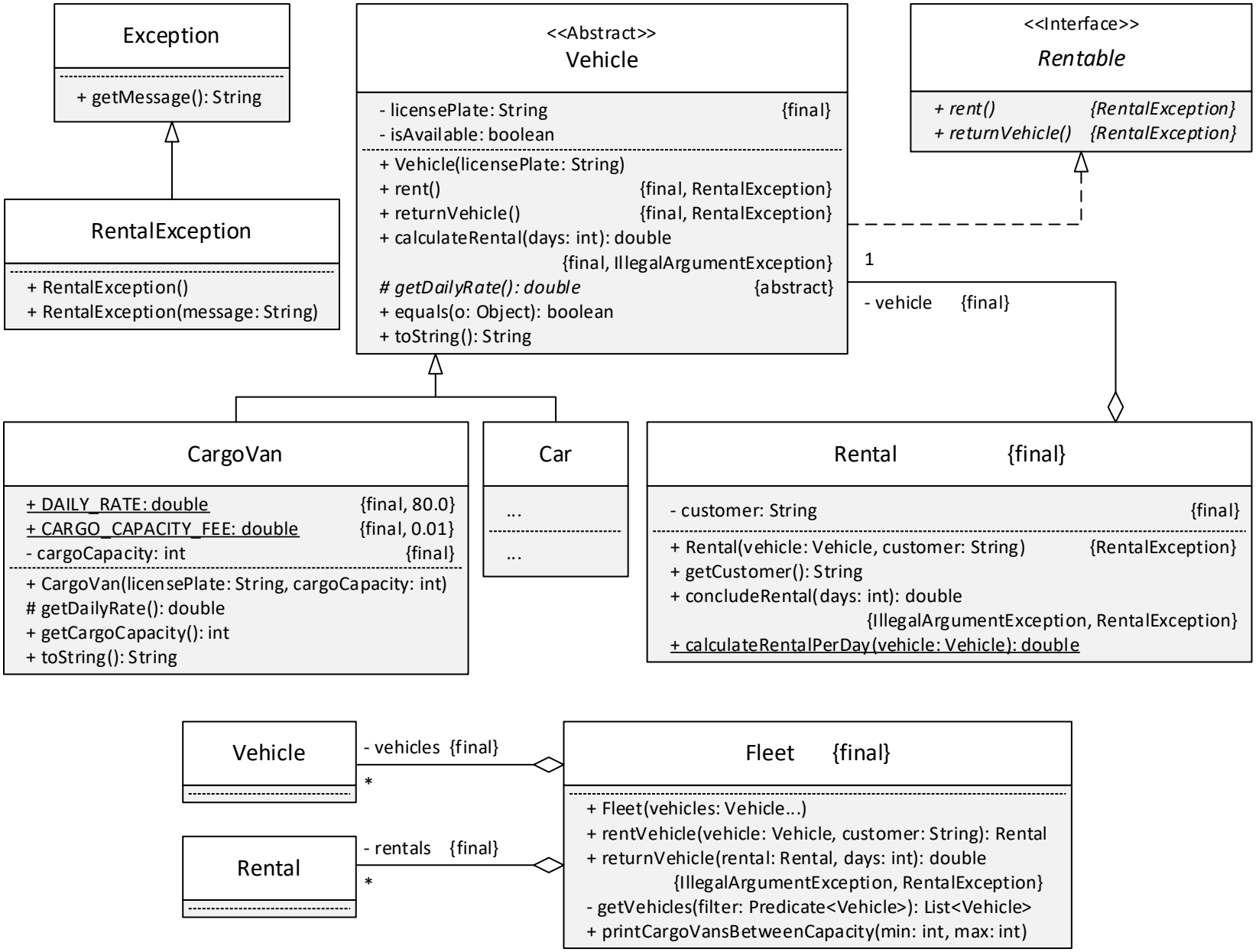
   - O campo **`cargoCapacity`** define a capacidade de carga, em litros, e é estabelecido no construtor com o valor do parâmetro **`cargoCapacity`**;
   - A tarifa diária resulta da adição do valor definido pela constante **`DAILY_RATE`** com a taxa aplicada à capacidade de carga. A taxa é definida pela constante **`CARGO_CAPACITY_FEE`**. Por exemplo, para um veículo com capacidade de carga de 1000 litros, a tarifa diária corresponde a 90€, ou seja, 80€ base mais 1% de 1000L;
   - O método **`getCargoCapacity`** é o método *accessor* para o campo **`cargoCapacity`**;
   - O método **`toString`** para um veículo com matrícula "PG-33-GP", disponível para alugar e com 3000L de capacidade de carga, produz a *string* **`CargoVan{cargoCapacity=3000, Vehicle{licensePlate="PG-33-GP", isAvailable=true}}`**. Na implementação deste método, deve considerar a *string* produzida pelo método **`toString`** da classe base.

3. [4] Implemente a classe **`Rental`**, que representa um contrato de aluguer. Um objeto deste tipo é criado quando o aluguer é iniciado e permanece válido até ao momento em que o aluguer é concluído. Considere os seguintes requisitos:

   - Na construção de um objeto **`Rental`**, o veículo indicado é alugado através da operação **`rent`**, sendo guardadas as referências para o veículo e para o nome do cliente. Se o veículo já estiver alugado, a construção lança **`RentalException`**;
   - O método **`getCustomer`** é o método *accessor* para o campo **`customer`**;
   - O método **`concludeRental`** finaliza o contrato de aluguer. Para tal, calcula o custo total do aluguer para o número de dias recebido no parâmetro **`days`** e, se o cálculo tiver sucesso, coloca o veículo novamente disponível para alugar e retorna o custo calculado. Se **`days`** for negativo, lança **`IllegalArgumentException`**, sem alterar o estado do veículo. Se o veículo já estiver disponível, lança **`RentalException`**;
   - O método **`calculateRentalPerDay`** retorna o custo de alugar por 1 dia o veículo especificado no parâmetro **`vehicle`**.

4. [8] Implemente a classe **`Fleet`** que representa uma frota de veículos. Considere os seguintes requisitos:

   - A frota de veículos agrega duas listas: (1) lista com todos os veículos que pertencem à frota (**`vehicles`**), independentemente de estarem alugados ou não; (2) lista com os contratos de aluguer ativos (**`rentals`**). Na representação das listas, utilize a interface **`List`** e a classe **`ArrayList`**;
   - Na criação de uma frota é estabelecido o conjunto de veículos que lhe pertence através do parâmetro do construtor **`vehicles`** e iniciada uma lista de contratos vazia. Considere que no conjunto de veículos indicado não existem matrículas repetidas nem veículos alugados;
   - O método **`rentVehicle`** aluga o veículo especificado no parâmetro **`vehicle`** ao cliente especificado no parâmetro **`customer`**. De notar que o objeto referido por **`vehicle`** pode não constar na lista de veículos, porém, terá de existir um objeto na lista que seja igual para que a operação tenha sucesso. Entenda-se "igual", uma instância de veículo que tenha a mesma matrícula que o veículo referido por **`vehicle`**. O contrato deve referenciar o objeto existente na lista da frota, mesmo que este seja uma instância diferente da recebida no parâmetro. Em caso de sucesso, o método adiciona o novo contrato à lista de contratos e retorna-o. Se não existir na frota um veículo igual ao indicado, ou se o veículo encontrado já estiver alugado, o método retorna **`null`**, sem adicionar qualquer contrato;
   - O método **`returnVehicle`** processa a conclusão do contrato de aluguer recebido no parâmetro **`rental`**. Se o contrato recebido não constar da lista de contratos ativos, o método lança **`RentalException`** com a mensagem *"Rental failed"*, sem alterar qualquer estado. Caso contrário, conclui o aluguer e, em caso de sucesso, remove o contrato da lista e retorna o custo total. Se a conclusão lançar uma exceção, o contrato mantém-se na lista;
   - O método **`getVehicles`** retorna uma nova lista com todos os veículos da frota que passem no filtro especificado pelo predicado **`filter`**;
   - O método **`printCargoVansBetweenCapacity`** escreve no *standard output* todos os veículos **`CargoVan`** que tenham capacidade de carga dentro do intervalo especificado pelos parâmetros **`min`** e **`max`**, inclusive. Cada **`CargoVan`** é impresso numa linha. Na implementação deste método deve recorrer ao método **`getVehicles`**, definindo o predicado por via de uma expressão *lambda*.


Exception
+ getMessage(): String
RentalException
+ RentalException()
+ RentalException(message: String)
<<Abstract>>
Vehicle
- licensePlate: String {final}
- isAvailable: boolean
+ Vehicle(licensePlate: String)
+ rent() {final, RentalException}
+ returnVehicle() {final, RentalException}
+ calculateRental(days: int): double
{final, IllegalArgumentException}
getDailyRate(): double {abstract}
+ equals(o: Object): boolean
+ toString(): String
<<Interface>>
Rentable
+ rent() {RentalException}
+ returnVehicle() {RentalException}
1
- vehicle {final}
CargoVan
+ DAILY_RATE: double {final, 80.0}
+ CARGO_CAPACITY_FEE: double {final, 0.01}
- cargoCapacity: int {final}
+ CargoVan(licensePlate: String, cargoCapacity: int)
getDailyRate(): double
+ getCargoCapacity(): int
+ toString(): String
Car
...
...
Rental {final}
- customer: String {final}
+ Rental(vehicle: Vehicle, customer: String) {RentalException}
+ getCustomer(): String
+ concludeRental(days: int): double
{IllegalArgumentException, RentalException}
+ calculateRentalPerDay(vehicle: Vehicle): double
Vehicle
- vehicles {final}
*
Rental
- rentals {final}
*
Fleet {final}
+ Fleet(vehicles: Vehicle...)
+ rentVehicle(vehicle: Vehicle, customer: String): Rental
+ returnVehicle(rental: Rental, days: int): double
{IllegalArgumentException, RentalException}
- getVehicles(filter: Predicate<Vehicle>): List<Vehicle>
+ printCargoVansBetweenCapacity(min: int, max: int)